

# ROS2

## DAY 1 — MANUAL 1

### (ROS2 THEORY + FOUNDATIONS + LINUX BASICS)

Complete Beginner-Friendly

---

## Chapter 1 — Introduction to Robotics Software

### 1.1 What is software in robotics?

Every robot needs three things:

1. **Sensors** → IMU, camera, LiDAR, etc.
2. **Actuators** → motors, servos
3. **A brain** → the software that connects everything

The "brain" must:

- read sensor data
- understand what's happening
- decide what the robot should do
- send commands to motors

Writing all this manually becomes extremely complicated.

That is why **ROS (Robot Operating System)** was created.

---

## Chapter 2 — What is ROS?

### 2.1 The biggest misunderstood fact

**ROS is NOT an operating system.**

It is a:

- **framework**
- **toolset**

- **collection of libraries**
- **communication system**

that helps you build robot software faster.

## 2.2 Why was ROS created?

Without ROS, every roboticist must manually write:

- communication between programs
- sensor drivers
- motor control logic
- LiDAR and IMU processing
- map building
- navigation
- multi-robot communication

This takes **months or years**.

ROS gives you **ready-made tools**, so you can focus on building your robot logic.

---

# Chapter 3 — What is ROS2 and why was it built?

## 3.1 ROS2 is the upgraded version of ROS1

ROS1 had limitations:

- poor real-time performance
- bad multi-robot communication
- poor security
- not suitable for industry
- depended on old Python 2
- not multi-platform

ROS2 solved everything.

## 3.2 ROS2 Features

| Feature                 | Explanation                               |
|-------------------------|-------------------------------------------|
| DDS-based communication | Fast, reliable, industry grade            |
| Real-time safe          | Can be used in drones, robots, automation |
| Multi-platform          | Runs on Linux, Windows, Mac               |

|                     |                                              |
|---------------------|----------------------------------------------|
| Multi-robot support | Robots can discover each other automatically |
| More secure         | Encrypted communication                      |
| Modern              | Supports Python 3 and C++17                  |

### 3.3 ROS2 = INDUSTRIAL-READY

Companies like:

- Boston Dynamics
- Clearpath
- iRobot
- Amazon Robotics

use ROS2.

---

## Chapter 4 — The ROS2 Architecture

To understand ROS2, you must understand 6 core concepts.

---

### 4.1 Node

A **node** is a small program that performs one task.

Examples:

- camera reader node
- imu processing node
- motor control node
- navigation node
- mapping node

✦ **Each node is like one app on your phone.**

Your robot = many apps (nodes) working together.

---

### 4.2 Topic

A **topic** is a channel used to send messages continuously.

Examples:

- `/cmd_vel` → velocity commands
- `/laser_scan` → LiDAR data
- `/imu/data` → IMU readings

! Topics are **one-way** communication.

**Publishers** send messages

**Subscribers** receive messages

---

## 4.3 Message

A message is a data structure sent over a topic.

Example:

`geometry_msgs/Twist` → command velocity

`std_msgs/String` → text message

---

## 4.4 Service

Used for **request–response** communication.

Example:

Node A → “Calculate path”

Node B → “Here is the path.”

---

## 4.5 Action

Used for **long tasks** requiring feedback.

Examples:

- robot navigating to a goal
- drone going to a waypoint
- robotic arm moving to a position

---

## 4.6 Package

A **package** contains everything needed to run a robot program:

- your node code
- setup files
- configuration files
- launch files (optional)

Think of it as a **folder containing all parts of a robot feature**.

---

## Chapter 5 — Workspaces

A **workspace** is like a big folder where all your ROS2 packages live.

Structure:

```
ros2_ws/  
  src/      ← your packages  
  install/  ← generated after build  
  build/    ← internal build data  
  log/      ← logs
```

You always create your packages in `src/`.

Then you build everything using:

```
colcon build
```

And activate the workspace:

```
source install/setup.bash
```

---

## Chapter 6 — Understanding Frameworks vs Libraries

(You asked for this specifically)

### 6.1 Library

- You call the library functions.

- You control the program.

Example:

```
import math
math.sqrt(16)
```

You are the boss.

---

## 6.2 Framework

- The framework controls the flow.
- You only write small parts (callbacks).
- The framework runs the entire system.

Example (ROS2):

- You write `timer_callback()`
- But ROS2 calls it repeatedly.
- ROS2 controls communication and execution.

✦ **Library = You call it**

✦ **Framework = It calls you**

---

# Chapter 7 — Linux Fundamentals for ROS2

ROS2 requires Linux basics.

Let's understand each command simply.

---

## 7.1 sudo

### Super User DO

Runs a command with administrator privileges.

Example:

```
sudo apt install python3
```

Analogy:  
You need the “master key” to install software.

---

## 7.2 apt

### **Advanced Package Tool**

Used for installing, removing, updating software.

Examples:

```
sudo apt update
sudo apt install curl
sudo apt remove firefox
```

---

## 7.3 curl

A tool that **downloads files from the internet**.

Example:

```
curl -O https://example.com/file.txt
```

Analogy:  
curl = a downloader.

---

## 7.4 chmod +x

Makes a file **executable** (like turning on permission to run it).

```
chmod +x simple_pub.py
```

Analogy:  
Giving someone “permission to drive a vehicle”.

---

## 7.5 mkdir

Creates folders.

```
mkdir ros2_ws
```

---

## 7.6 ls

Lists files.

```
ls -l
```

---

## 7.7 cd

Changes directory.

```
cd ros2_ws/src
```

---

## 7.8 source

Loads configuration into the current terminal.

```
source ~/.bashrc
```

---

# Chapter 8 — DDS (Why ROS2 is Fast)

Beginner-friendly explanation.

DDS = **Data Distribution Service**

Purpose:

Allows nodes to communicate without a master server.

Features:

- auto discovery
- fast
- reliable
- configurable reliability
- security

Example:

Two robots meet → they instantly discover each other's topics without setup.

---



# Chapter 9 — Summary of Everything Learned

You now understand:

- What is ROS & ROS2
- Why ROS2 is needed
- Nodes, topics, services, actions, messages
- What are packages
- What is a workspace
- Framework vs library
- Basic Linux commands used in ROS2
- DDS basics

This completes **Manual 1** (Theory).

---